FSD - Harnessing Gen AI: From Design to Code Optimization

Generative AI-Assisted Code Generation

Engage and Think

Imagine you are a software developer working on an online learning platform. You realize the project requires features like real-time student feedback and adaptive testing algorithms. To manage the coding workload efficiently, you integrate Generative AI into your development process.

How can Generative AI assist in streamlining code generation?

Learning Objectives

By the end of this lesson, you will be able to:

- Make use of Generative AI platforms to generate code based on user inputs or project requirements automatically
- Utilize GitHub Copilot to automate the boilerplate code generation
- Use GenAI-enabled GitHub Copilot to generate key components of a Flask application, including routes, views, and templates
- Create customized code using Generative AI tools for code completion and suggestion

Leveraging Generative AI for Code Generation

Phases in Software Development

After utilizing Generative AI in the design phase, the next step is to understand the implementation phase and how Generative AI can be integrated into it.

Project planning

Requirement gathering and analysis

Designing

Implementing

Testing

Deployment

Support and maintenance

Implementation Phase in Software Development

In this phase of SDLC, software source code is written based on the design specifications created in the earlier stages.

Generative AI can be used in this phase to bridge the gap between design and implementation by automatically generating code from design specifications.

Automated Code Generation (ACG)

It refers to the process of automatically generating source code or other code components using different tools, frameworks, or techniques.

The objective is to optimize efficiency, reduce human error, and accelerate software development by automating repetitive and monotonous tasks.

Automated Code Generation Using Generative AI

Generative AI (GenAI) models and tools play a transformative role in automated code generation by leveraging machine learning and natural language processing techniques.

They generate code by leveraging machine learning models, particularly large language models (LLMs) like GPT, which have been trained on vast programming languages and code repositories.

Popular Generative AI Tools for Code Generation

GitHub Copilot

AI-powered code completion tool providing real-time suggestions in the editor

Amazon Q

AI tool by Amazon delivering real-time code suggestions and completions

ChatGPT

Conversational AI by OpenAI capable of assisting with various tasks, including code generation

Popular Generative AI Tools for Code Generation

Copilot

Conversational AI-powered tool to provide real-time code suggestions, autocompletion, and error handling.

Gemini

Gemini features multimodal capabilities, blending text, images, and context for advanced AI applications.

tabnine

AI-driven tool for predicting and suggesting code completions in the IDE.

How Does Generative AI Generate Code?

The following are techniques that help Generative AI models generate code:

Training on large codebases	They are trained on large code bases from open repositories, software libraries, and documentation, allowing them to learn code structure and syntax.
Predictive coding	Once trained, they can predict the next sequence of code based on a given input or prompt.
Contextual understanding	They can process the entire code, keeping track of variables, functions, and logic to provide more accurate code suggestions, autocompletion, and generation.

How Does Generative AI Generate Code?

Natural language to code translation

These models can convert natural language prompts to generate code using natural language processing (NLP).

Learning from feedback

Generative AI tools allow developers to interact through feedbacks, which help them to improve their code output.

This combination of training, prediction, and contextual understanding allows Generative AI to assist in code generation, significantly enhancing developer productivity.

Industries Using Generative AI

Various industries globally are leveraging Generative AI technology to generate code for different operations, such as:

Healthcare	Automates medical coding and diagnosis, enhancing efficiency in healthcare operations
Finance	Streamlines tasks like fraud detection and generates personalized investment portfolios
Retail	Enhances customer experiences by creating personalized product recommendations
Software	Automates repetitive tasks like generating code and unit tests for increased productivity

Enhancing Coding Efficiency with Generative AI

Generative AI offers a range of capabilities to streamline and enhance the coding process, including:

Code suggestions

Provides real-time code suggestions and auto-completion

Automated code generation

Generates boilerplate code, unit tests, documentation, and complex algorithms

Error detection and debugging

Helps in identifying potential bugs and suggests fixes

Customization

Fine-tune for specific coding styles or project needs

Quick Check

As a software developer, you often write repetitive code, which takes time. To improve efficiency, you decide to use a Generative AI tool. Which capability of Generative AI would best reduce the time you spend on these repetitive tasks?

A. Code suggestions

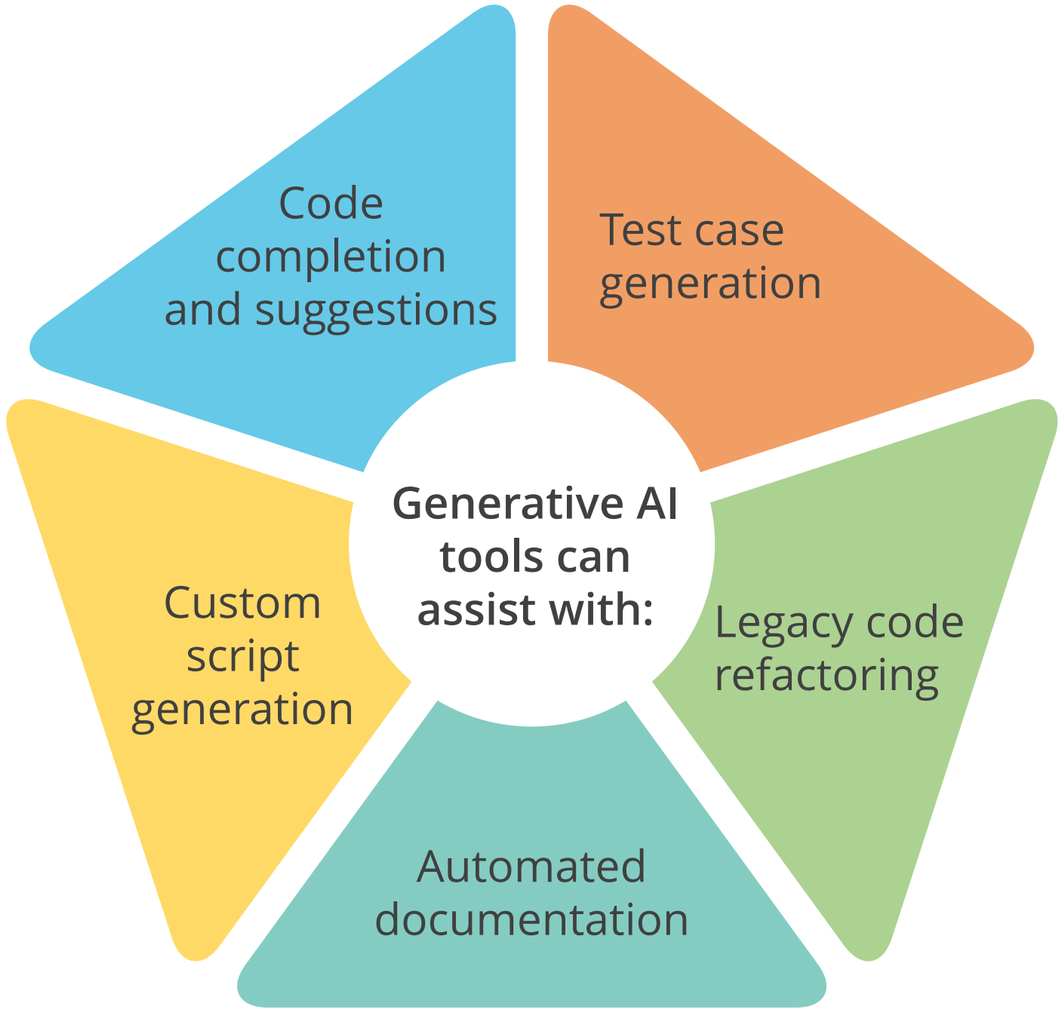
B. Automated code creation

C. Error detection and debugging

D. Customization

Generative AI: Transforming Code Generation

How Generative AI Transforms Code Generation



Code Completion and Suggestions

GenAI tools provide real-time coding assistance in integrated development environments (IDEs) by suggesting how to complete the code based on the current context.

The key benefits include:

01

Increased productivity
Increases coding speed by automatically completing and proposing code fragments

02

Error reduction
Minimizes typos and syntax errors with insightful recommendations

03

Enhanced learning
Aids developers in comprehending the effective usage of new libraries and APIs

Code Completion and Suggestions: Example

This feature of Generative AI tools helps developers quickly implement the logic without manually writing entire blocks of code.

Example

Task: Develop a basic login system where users input their credentials to gain access to personalized features

AI assistance: The GenAI tool generates a Python **login(username, password)** function that checks if the credentials match predefined values. If they match, it returns **True** to grant access; otherwise, it returns **False**.

```
# Code for simple login functionality
                                  1/1  Accept Tab  Accept Word Ctrl + RightArrow  ...
def login(username, password):
    if username == "admin" and password == "admin":
        return True
    else:
        return False
```

Here is code that GitHub Copilot autogenerates or suggests when it detects the function name as login. Once accepted, the code is placed in the file.

Demo: Building a To-Do List Application Using GitHub Copilot

Duration: 10 minutes

Demonstrate how to build a to-do list application using GitHub Copilot

You are overseeing the development of a To-Do List application using Python Flask, supported by GitHub Copilot in Visual Studio. This project leverages generative AI to enhance the efficiency of backend API creation essential for task management applications.

The integration of GitHub Copilot significantly accelerates the development process by providing real-time code suggestions and completions. This tool facilitates the rapid generation of robust and scalable APIs, which include capabilities for task addition, prioritization, categorization, and scheduling.

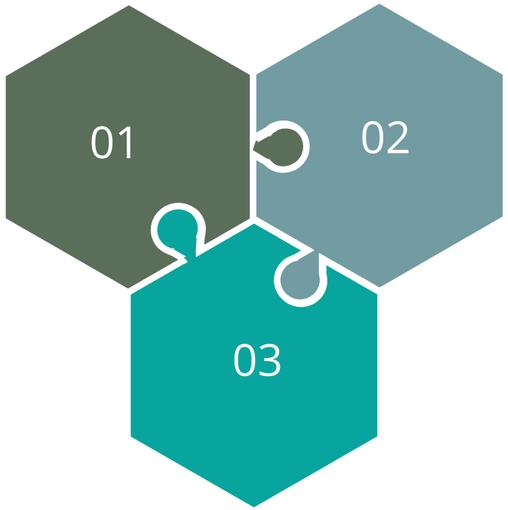
DEMONSTRATION

Note: Refer to the demo document for detailed steps:
01_Building_a_To_Do_List_Application_Using_GitHub_Copilot

Custom Script Generation

Generative AI automatically creates tailored code snippets based on specific user requirements or predefined inputs. The key benefits include:

Task automation
Streamlines repetitive and complex tasks



Customization
Tailors scripts to specific user needs and contexts

Efficiency
Minimizes time and effort in manual script writing

Custom Script Generation

Custom script generation using Generative AI include:

Boilerplate code generation	GenAI can generate standard code structures known as boilerplate code, ensuring faster coding for routine operations.
APIs and libraries integration	It can recommend and integrate specific APIs or libraries based on the project's requirements.
Error handling and debugging support	It can handle common errors and debug them while generating the script, ensuring its robustness.

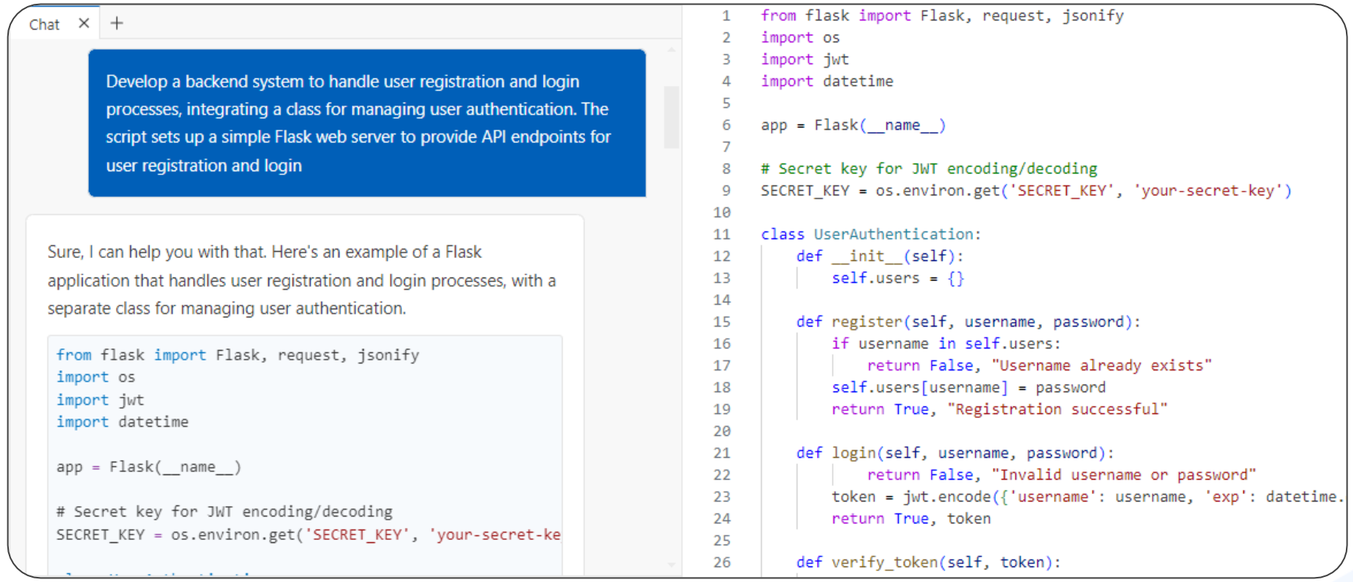
Custom Script Generation: Example

Generative AI tools enable rapid and efficient script creation by translating natural language prompts into fully functional code for various software features.

Example

Task: Develop a backend system for user registration and login with a class for user authentication, using a Flask web server to provide API endpoints

AI assistance: Generative AI generates a custom Flask web app script with API endpoints for user registration and login based on the provided prompt.



Demo: Building a Grocery App with React GitHub Copilot and JSON Server

Duration: 10 minutes

Demonstrate how to build a Grocery App using GitHub Copilot, React JS, and JSON

You are overseeing the development of a Grocery App using React JS, and JSON Server, supported by GitHub Copilot in Visual Studio. This project aims to leverage generative AI to construct REST APIs and Postman for comprehensive testing and interaction.

Building these APIs with GitHub Copilot speeds up development and ensures that they are robust and scalable. It fetches and displays products, users, and orders from a mock backend, allowing you to interact with the data through a simulated REST API.

DEMONSTRATION

Note: Refer to the demo document for detailed steps:
Demo_02_Building_a_Grocery_App_with_React_GitHub_Copilot_and_JSON_Server

Automated Documentation

It entails the generation of comments, documentation, and explanations for code automatically. The following are key benefits of this application:

1. **Time efficiency**
   Streamlines the process of creating documentation

2. **Consistency**
   Maintains uniformity in documentation standards throughout the project

3. **Improved understanding**
   Facilitates faster understanding of the codebase for new developers

Demo: Automating Boilerplate Code Generation with GitHub Copilot

Duration: 10 minutes

Demonstrate how to automate boilerplate code generation with GitHub Copilot

You are coordinating the development of a web service using GitHub Copilot to automate boilerplate code generation in a college management system, focusing specifically on creating standard API endpoints to manage educational department and student data, demonstrating the efficiency of this tool in streamlining the development process.

GitHub Copilot facilitates the rapid boilerplate code generation for the React application, significantly expediting the development process. This approach boosts productivity, minimizes errors, and allows developers to focus on more complex aspects of project development.

Note: Refer to the demo document for detailed steps:
03_Automating_Boilerplate_Code_with_GitHub_Copilot

DEMONSTRATION

Legacy Code Refactoring

It is the process of updating and improving outdated or legacy software systems using AI techniques. The following are key benefits of this application:

Key Benefits

01 Code quality improvement
Enhances readability, maintainability, and performance

02 Reduced technical debt
Updates outdated code practices and dependencies

03 Risk mitigation
Identifies and resolves potential issues in legacy code

Legacy Code Refactoring: Example

Generative AI tools can streamline the process of legacy code refactoring, enhancing efficiency and maintainability. The following example shows the enhancement of an unstructured code:

Problem

In an existing software project, there's a need to refactor legacy code that handles user authentication. The current system lacks scalability and security measures and does not follow modern best practices for user management.

Additionally, there is no existing mechanism for handling multiple user roles or permissions. The given code is seen before Generative AI performs refactoring.

```python
# Code for simple login functionality

def login(username, password):
    if username == "admin" and password == "admin":
        return True
    else:
        return False

    # A dictionary to store registered users
users = {}

def register(username, password):
    if username in users:
        return False
    else:
        users[username] = password
        return True
```

Legacy Code Refactoring: Example

Refactoring legacy code using Generative AI can significantly enhance the unstructured code into the structured.

Solution

This refactored code, improved using Generative AI tools, enhances the maintainability and security of authentication systems in software applications by ensuring modern coding practices and comprehensive documentation.

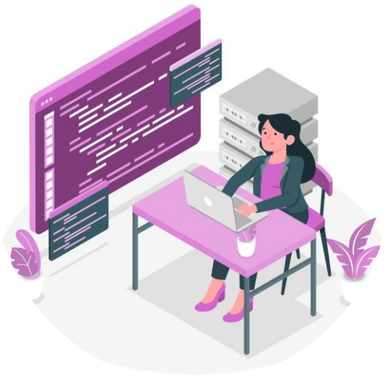
```python
class User:
    def __init__(self, username, password):
        self.username = username
        self.password = password

class LoginSystem:
    def __init__(self):
        self.users = {}

    def login(self, username, password):
        if username in self.users and self.users[username].password == password:
            return True
        else:
            return False

    def register(self, username, password):
        if username in self.users:
            return False
        else:
            self.users[username] = User(username, password)
            return True
```

Test Case Generation

It is the process of automatically creating unit tests for software components.
The following are key benefits of this application:

Code reliability

Ensures that code functions as expected and enhances overall software quality



Coverage

Improves test coverage by identifying edge cases and potential bugs

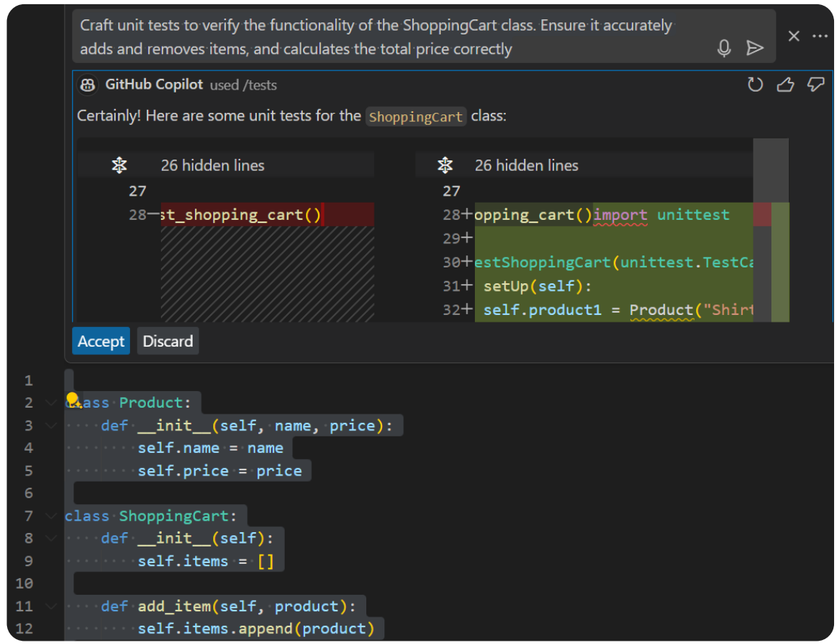
Test Case Generation: Example

Generative AI automatically analyzes code structures and generates diverse test scenarios, ensuring comprehensive test coverage.

Example

Task: Develop unit tests for a **ShoppingCart** class using Generative AI

AI assistance: Generative AI analyzes the **ShoppingCart** class and generates unit tests for key functions, covering edge cases, validating inputs, ensuring accurate price calculations, and streamlining the testing process.



Demo: Generating Component Architecture and Building a Structured React Blog Using GitHub Copilot

Duration: 10 minutes

DEMONSTRATION

Demonstrate how to generate component architecture and build a structured React blog using GitHub Copilot

You are overseeing the creation of a structured React blog application, utilizing GitHub Copilot to generate component architecture and streamline code development within Visual Studio Code.

GitHub Copilot is employed to outline and develop the core architecture of the React application. Starting with generating the architecture, you specify the arrangement of components such as App.js, BlogList.js, and BlogPost.js within a structured folder setup, including a styling directory for App.css.

Note: Refer to the demo document for detailed steps:
04_Generating_Component_Architecture_and_Building_a_Structured_React_Blog_Using_GitHub_Copilot

Demo: Using a Pretrained Model to Create an Application Structure

Duration: 10 minutes

Demonstrate how to use a pretrained model to create an application structure

You are managing the development of a dynamic code generation tool that creates directory structures for various types of applications, utilizing a hypothetical machine learning model called MyStructureGenModel.

This approach streamlines the setup process for various application projects and showcases the practical application of integrating machine learning models with code generation tools to enhance development workflows and reduce manual setup tasks.

DEMONSTRATION

Note: Refer to the demo document for detailed steps:
05_Using_a_Pre_trained_Model_to_Create_an_Application_Structure

Quick Check

A development team is building a new feature for their application and wants to reduce the time spent writing repetitive code while ensuring comprehensive documentation is maintained. Which capability of Generative AI would be most helpful in streamlining this process?

A. Generating test case for edge cases

B. Generating custom script from scratch

C. Automating boilerplate code generation

D. Refactoring legacy code for efficiency

Challenges of Generative AI in Code Generation

Challenges: Code Complexity

Generative AI faces several challenges in understanding and creating complex code. The following are some of the key issues:

Error propagation
It may generate incorrect or suboptimal code, propagating errors.

Context understanding
It struggles to grasp intricate, project-specific contexts.

Complex logic
It encounters challenges in handling intricate logic and dependencies in large codebases.

Challenges: Quality and Reliability

It ensures that code generation consistently produces reliable and consistent outcomes.

Reliability

Guarantee dependable code performance under varying conditions

Consistency

Maintain uniform code quality across diverse modules and projects

Testing

Rigorously test AI-generated code to meet established quality standards

Challenges: User Adoption

It refers to the impediments hindering the developers' extensive acceptance and utilization of a technology or tool. The key issues are:

Trust and skepticism

Developers may question the trustworthiness of AI-generated solutions

Learning curve

Significant dedication of time and effort is necessary to master the utilization of AI tools

Integration

Difficulties arise in seamlessly incorporating AI tools into established development flow

Use Cases: Generative AI in Code Generation

Use Case: Utilizing Generative AI to Save Time and Cost

- The Thoughtworks team has used Generative AI to increase productivity throughout the software development lifecycle.
- However, quantifying Generative AI's value has been challenging.
- This leads to a narrow focus on coding productivity and neglects its potential impact on complex, waste-prone areas of development.

/thoughtworks

Problem

Use Case: Utilizing Generative AI to Save Time and Cost

- By integrating Generative AI into a comprehensive toolset, Thoughtworks has enhanced productivity, reduced time to market, improved quality, and maintained team morale.
- Their experiments with tools like ChatGPT and GitHub Copilot have shown a 10-30% increase in productivity, especially when experienced developers apply these tools to well-defined problems.

ChatGPT

GitHub Copilot

Problem

Use Case: Maximizing Software Development Productivity with Generative AI

Problem

The Accenture Velocity team is looking for ways to improve developer productivity and efficiency in their software engineering practices.

Solution

- Accenture uses Amazon CodeWhisperer to speed up coding in their Velocity platform, reducing development efforts by up to 30% and allowing focus on security, quality, and performance.
- CodeWhisperer provides real-time code recommendations, writes boilerplate code, helps with unfamiliar languages, and detects security vulnerabilities early.

accenture

Amazon CodeWhisperer

Comparing AI Code Generation Tools: GitHub Copilot, Amazon Q, and ChatGPT

Research Objective

Scenario

AI-assisted code generation tools are increasingly used in software engineering to generate code from natural language prompts or partial code inputs. Notable examples include GitHub Copilot, Amazon Q, and ChatGPT.

Objective

This study compares the performance of these tools based on various code quality metrics such as:

- Code validity
- Code correctness
- Code security
- Code reliability
- Code maintainability

Amazon Q

ChatGPT

GitHub Copilot

Research Results

The study highlights the following strengths and weaknesses of these code-generation GenAI tools, providing valuable insights for practitioners and enhancing the decision-making processes:

Tools	Code validity (%)	Code security (%)	Code reliability (%)	Code maintainability (%)
ChatGPT	65.2	73	75	68
GitHub Copilot	46.3	80	70	60
Amazon Q	31.1	85	68	65

Guided Practice

Overview

Duration: 25 minutes

In this exercise, we created a basic React application with the assistance of GitHub Copilot, focusing on developing a Tic Tac Toe game.

The process was divided into Project Setup, UI Creation, Game Logic, Bug Fixes, and UI Enhancement. Throughout the project, GitHub Copilot acted as a coding assistant, providing code suggestions and helping to streamline the development process.

The use of AI tools significantly expedited workflows and facilitated successful project completion.

Guided Practice

Duration: 25 minutes

Tasks:

Initialize the GitHub Copilot project to create a React sample project:

1. Set up the workspace and open the generated React project
2. Ensure the project environment is configured properly to create a basic Tic Tac Toe game using GitHub Copilot
3. Use prompts to create the basic user interface
4. Use prompts to develop the basic game logic and determine when a player wins
5. Use prompts to identify and fix bugs
6. Use prompts to enhance the board's user interface aesthetics

Key Takeaways

- Generative AI tools for code generation include GitHub Copilot, Amazon CodeWhisperer, and Tabnine, providing real-time code suggestions, completions, optimization, and generation capabilities.
- Generative AI enhances developer productivity and code quality through real-time assistance, automated documentation, code modernization, custom scripts, and unit test generation.
- Generative AI currently struggles with understanding and generating complex code, particularly in contextual comprehension, error handling, and managing intricate logic.

Practice Project

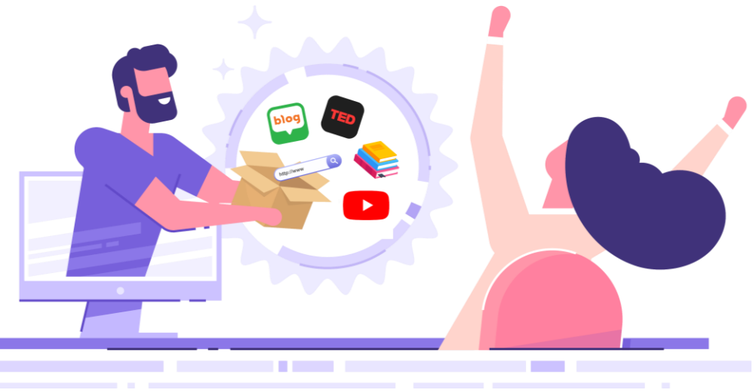
Generative AI Assisted Code Generation

The project aims to leverage GitHub Copilot and Visual Studio Code to develop a simple recipe app using React, showcasing a variety of recipes. The app will utilize mock sample data or generated content to populate the recipes. Key features include a list page to display all recipes, a detailed page for individual recipes, integration of images for each dish, and implementation of search and filter functionalities for enhanced user experience.

Tasks:

1. Set up the project
2. Create mock recipe data
3. Build the recipe list component
4. Implement routing with React router
5. Create the recipe detail component
6. Implement search functionality
7. Add filtering options
8. Style components
9. Debug common errors

Additional Resources



- AI code-generation software https://www.ibm.com/blog/ai-code-generation/
- Refer to this link for how AI code generation works https://github.blog/2024-02-22-how-ai-code-generation-works/
- Research on tools: **https://ar5iv.labs.arxiv.org/html/2304.10778**

Q&A